IFS 415 – Enterprise Architecture

INTRODUCTION

building an organisation is a complex and challenging task because of the multifarious dependencies within an organisation. Many (often unknown) dependencies exist between various domains, like strategy, products and services, business processes, organisational structure, applications, information management and technical infrastructure. Besides having a good overview of these different domains, one needs to be aware of their interrelationships. Together, these form the enterprise architecture of the organisation.

It is often said that to manage the complexity of any large organisation or system, you need architecture. Building or rebuilding an organisation is a much more complex and challenging task. First of all, the steps one has to take in order to (re)build an organisation are not standardised. One could start by first (re)designing business processes, followed by the application (re)design. Or one could first design generic application services, followed by designing business processes on top of these. Since a few years, The Open Group Architectural Framework (TOGAF) has defined a standard way to take these steps. This enables enterprise architects to (re)design an organisation and its supporting IT systems in a uniform and standard way. The release of the improved TOGAF 9 version in February 2009 will lead to an even more uniform and better way to do this.

This representation of the system's essence, also called the '**architecture'** of the system, forms the basis for analysis, optimisation, and validation and is the starting point for the further design, implementation, and construction of that system. The resulting artifacts, be they buildings or information systems, naturally have to conform to the original design criteria. The definition of the architecture is the input for verifying this.

But what exactly does 'architecture' mean? Of course, we have long known this notion from building and construction. Suppose you contract an architect to design your house. You discuss how rooms, staircases, windows, bathrooms, balconies, doors, a roof, etc., will be put together. You agree on a master plan, on the basis of which the architect will produce detailed specifications, to be used by the engineers and builders.

How is it that you can communicate so efficiently about that master plan? We think it is because you share a common frame of reference: you both know what a 'room' is, a 'balcony', a 'staircase', etc. You know their function and their relation. A 'room', for example, serves as a shelter and is connected to another 'room' via a 'door'. You both use, mentally, an architectural model of a house. This model defines its major functions and how they are structured. It provides an abstract design, ignoring man details. These details, like the number of rooms, dimensions, materials to be used, and colours, will be filled in later.

A similar frame of reference is needed in designing an enterprise. To create an overview of the structure of an organisation, its business processes, their application support, and the technical infrastructure, you need to express the different aspects and domains, and their relations.

But what is 'architecture' exactly?

Architecture is the fundamental organisation of a system embodied in its components, their relationships to each other, and to the environment, and the principle guiding its design and evolution.

This definition accommodates both the blueprint and the general principles. More succinctly, we could define architecture as 'structure with a vision'. An architecture provides an integrated view of the system being designed or studied. To begin with, architecture, and hence the architect, is concerned with understanding and defining the relationship between the users of the system and

the system being designed itself. Based on a thorough understanding of this relationship, the architect defines and refines the essence of the system, i.e., its structure, behaviour, and other properties. Most stakeholders of a system are probably not interested in its architecture, but only in the impact of this on their concerns. However, an architect needs to be aware of these concerns and discuss them with the stakeholders, and thus should be able to explain the architecture to all stakeholders involved, who will often have completely different backgrounds.

Stakeholder: an individual, team, or organisation (or classes thereof) with interests in, or concerns relative to, a system.

During this process, the architect needs to communicate with all stakeholders of the system, ranging from clients and users to those who build and maintain the resulting system. The architect needs to balance all their needs and constraints to arrive at a feasible and acceptable design. Fulfilling these needs confronts the methodology for defining and using architectures with demanding requirements. These can only be met if the architects have an appropriate way of specifying architectures and a set of design and structuring techniques at their disposal, supported by the right tools. In building and construction, such techniques and tools have a history over millennia. In information systems and enterprise architecture, though, they are just arising.

ENTERPRISE ARCHITECTURE

More and more, the notion of architecture is applied with a broader scope than just in the technical and IT domains. The emerging discipline of enterprise engineering views enterprises as a whole as purposefully designed systems that can be adapted and redesigned in a systematic and controlled way. An ‘enterprise’ in this context can be defined as follows: (The Open Group 2009a):

Enterprise: any collection of organisations that has a common set of goals and/or a single bottom line.

Architecture at the level of an entire organisation is commonly referred to as ‘enterprise architecture’. This leads us to the definition of enterprise architecture:

Enterprise architecture: a coherent whole of principles, methods, and models that are used in the design and realisation of an enterprise’s organisational structure, business processes, information systems, and infrastructure.

Enterprise architecture captures the essentials of the business, IT and its evolution. The idea is that the essentials are much more stable than the specific solutions that are found for the problems currently at hand.

Architecture is therefore helpful in guarding the essentials of the business, while still allowing for maximal flexibility and adaptivity. Without good architecture, it is difficult to achieve business success. The most important characteristic of an enterprise architecture is that it provides a holistic view of the enterprise. Within individual domains local optimisation will take place and from a reductionistic point of view, the architectures within this domain may be optimal. However, this need not lead to a desired situation for the company as a whole.

For example, a highly optimised technical infrastructure that offers great performance at low cost might turn out to be too rigid and inflexible if it needs to support highly agile and rapidly changing business processes. A good enterprise architecture provides the insight needed to balance these requirements and facilitates the translation from corporate strategy to daily operations. To achieve this quality in enterprise architecture, bringing together information from formerly unrelated domains necessitates an approach that is understood by all those involved from these different domains. In contrast to building architecture, which has a history over millennia in which a common language and culture has been established, such a shared frame of reference is still lacking in business and IT. In current practice, architecture descriptions are heterogeneous in

nature: each domain has its own description techniques, either textual or graphical, either informal or with a precise meaning. Different fields speak their own languages, draw their own models, and use their own techniques and tools. Communication and decision making across these domains is seriously impaired.

What is part of the enterprise architecture, and what is only an implementation within that architecture, is a matter of what the business defines to be the architecture, and what not. The architecture marks the separation between what should not be tampered with and what can be filled in more freely. This places a high demand for quality on the architecture. Quality means that the architecture actually helps in achieving essential business objectives. In constructing and maintaining an architecture, choices should therefore be related to the business objectives, i.e., they should be rational.

Even though an architecture captures the relatively stable parts of business and technology, any architecture will need to accommodate and facilitate change, and architecture products will therefore only have a temporary status. Architectures change because the environment changes and new technological opportunities arise, and because of new insights as to what is essential to the business. To ensure that these essentials are discussed, a good architecture clearly shows the relation of the architectural decisions to the business objectives of the enterprise.

The instruments needed for creating and using enterprise architectures are still in their infancy. To create an integrated perspective of an enterprise, we need techniques for describing architectures in a coherent way and communicating these with all relevant stakeholders. Different types of stakeholders will have their own viewpoints on the architecture. Furthermore, architectures are subject to change, and methods to analyse the effects of these changes are necessary in planning future developments. Often, an enterprise architect has to rely on existing methods and techniques from disparate domains, without being able to create the 'big picture' that puts these domains together. This requires an integrated set of methods and techniques for the specification, analysis, and communication of enterprise architectures that fulfils the needs of the different types of stakeholders involved.

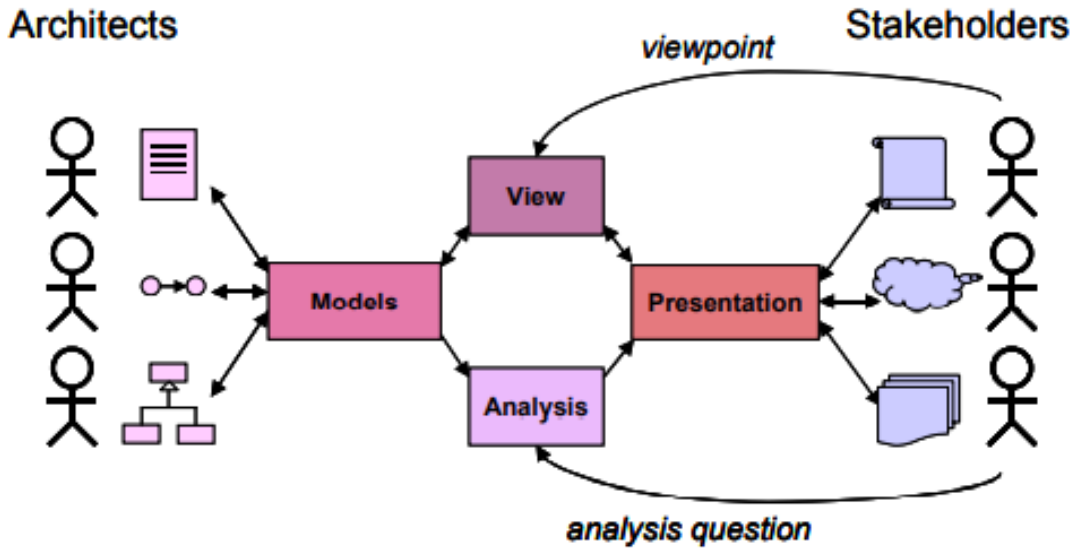

Fig. 1.1. Communicating about architecture.

THE ARCHITECTURE PROCESS

Architecture is a process as well as a product. The product serves to guide managers in designing business processes and system developers in building applications in a way that is in line with business objectives and policies. The effects of the process reach further than the mere creation of the architecture product – the awareness of stakeholders with respect to business objectives

and information flow will be raised. Also, once the architecture is created, it needs to be maintained. Businesses and IT are continually changing. This constant evolution is, ideally, a rational process. Change should only be initiated when people in power see an opportunity to strengthen business objectives.

The architecture process consists of the usual steps that take an initial idea through design and implementation phases to an operational system, and finally changing or replacing this system, closing the loop. In all of the phases of the architecture process, clear communication with and between stakeholders is indispensable. The architecture descriptions undergo a life cycle that corresponds to this design process (Fig. 1.2). The different architecture products in this life cycle are discussed with stakeholders, approved, revised, etc., and play a central role in establishing a common frame of reference for all those involved.

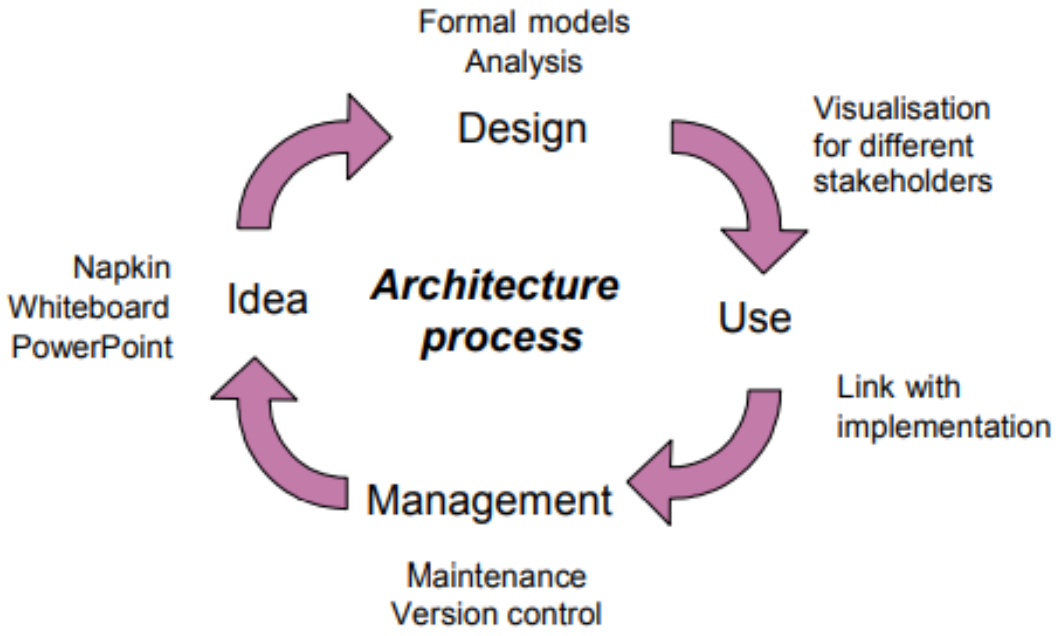

Fig. 1.2. The architecture description life cycle.

DRIVERS FOR ENTERPRISE ARCHITECTURE

It need not be stressed that any organisation benefits from having a clear understanding of its structure, products, operations, technology, and the web of relations tying these together and connecting the organisation to its surroundings.

INTERNAL DRIVERS

Business-IT alignment is commonly recognised as an important instrument to realise organisational effectiveness. Such effectiveness is not obtained by local optimisations, but is realised by well-orchestrated interaction of organisational components (Nadler et al. 1992). Effectiveness is driven by the relationships between components rather than by the detailed specification of each individual component. A vast amount of literature has been written on the topic of alignment, underlining the significance of both ‘soft’ and ‘hard’ components of an organisation. Parker and Benson (1989) were forerunners in using the term ‘alignment’ in this context and emphasising the role of architecture in strategic planning.

The well-known strategic alignment model of Henderson and Venkatraman (1993) distinguishes between the aspects of business strategy and organisational infrastructure on the one hand, and IT strategy and IT infrastructure on the other hand (Fig. 1.3). The model provides four dominant perspectives that are used to tackle the alignment between these aspects. One can take the business strategy of an enterprise as the starting point, and derive its IT infrastructure either via an IT strategy or through the organisational infrastructure; conversely, one can focus on IT as an enabler and start from the IT strategy, deriving the organisational infrastructure via a business

strategy or based on the IT infrastructure. In any of these perspectives, an enterprise architecture can be a valuable help in executing the business or IT strategy.

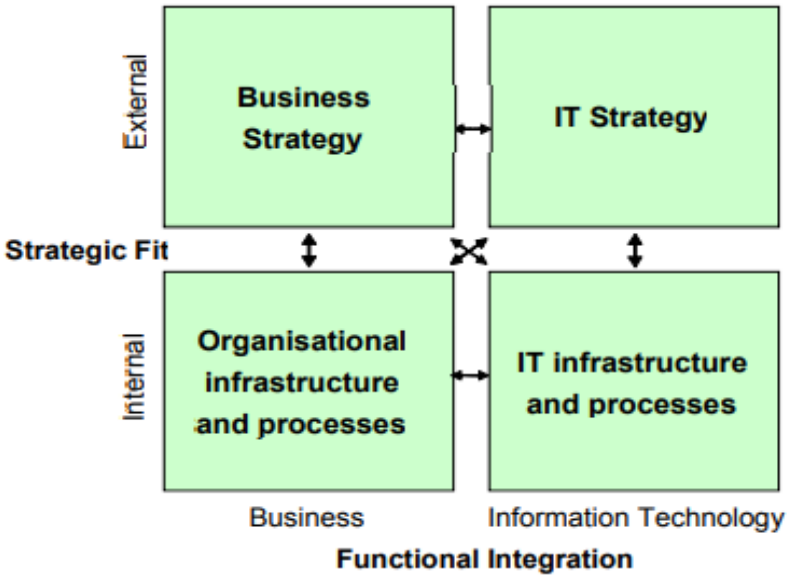

Fig. 1.3. Strategic alignment model (Henderson and Venkatraman 1993).

Nadler et al. (1992) identify four relevant alignment components: work, people, the formal organisation and the informal organisation.

Labovitz and Rosansky (1997) emphasise the horizontal and vertical alignment dimensions of an organisation. Vertical alignment describes the relation between the top strategy and the people at the bottom, whereas horizontal alignment describes the relation between internal processes and external customers. Obviously, the world of business-IT alignment is as diverse as it is complex.

In coping with this complexity, enterprise architecture is of valuable assistance. In Fig. 1.4, enterprise architecture is positioned within the context of managing the enterprise. At the top of this pyramid, we see the mission of the enterprise: why does it exist? The vision states its 'image of the future' and the values the enterprise holds. Next there is its strategy, which states the route the enterprise will take in achieving this mission and vision. This is translated into concrete goals that give direction and provide the milestones in executing the strategy. Translating those goals into concrete changes to the daily operations of the company is where enterprise architecture comes into play. It offers a holistic perspective of the current and future operations, and on the actions that should be taken to achieve the company's goals. Next to its architecture, which could be viewed as the 'hard' part of the company, the 'soft' part, its culture, is formed by its people and leadership, and is of equal if not higher importance in achieving these goals. Finally, of course, we see the enterprise's daily operations, which are governed by the pyramid of Fig. 1.4.

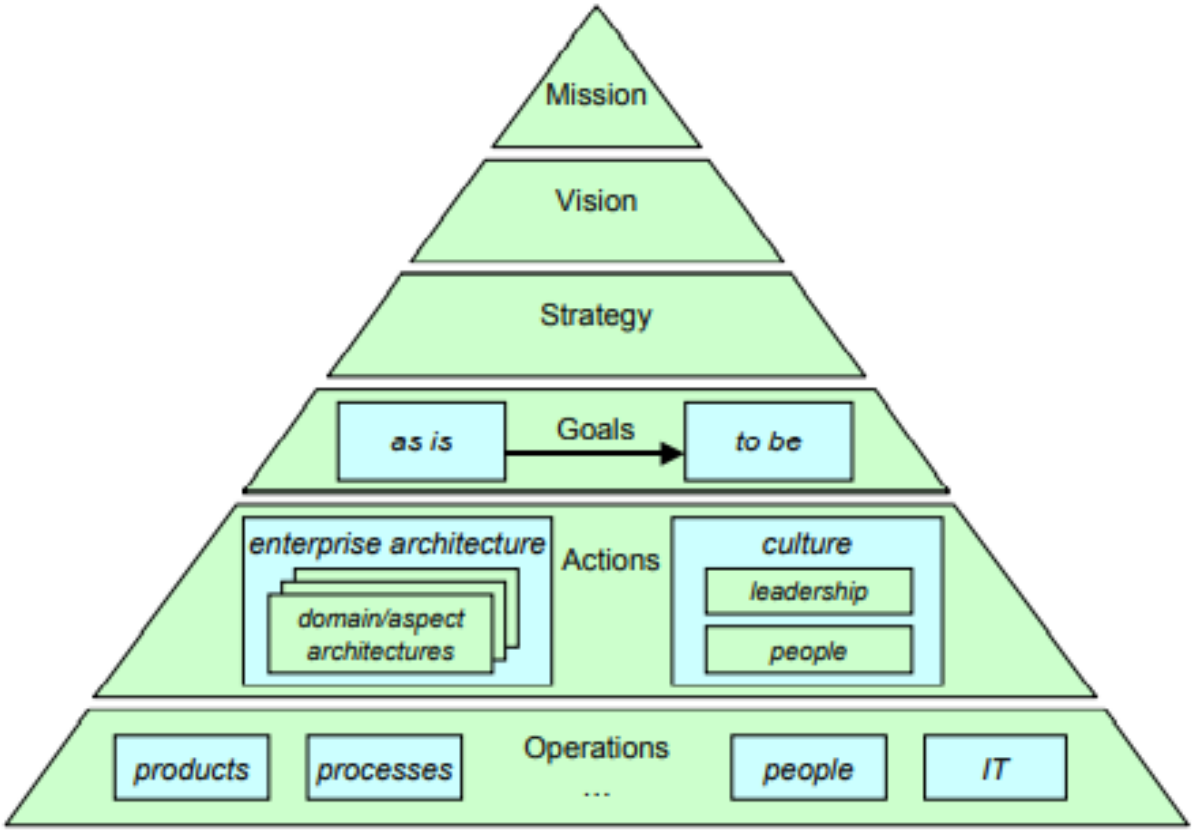

Fig. 1.4. Enterprise architecture as a management instrument.

To some it may seem that architecture is something static, confining everything within its rules and boundaries, and hampering innovation. This is a misconception. A well-defined architecture is an important asset in positioning new developments within the context of the existing processes, IT systems, and other assets of an organisation, and it helps in identifying necessary changes. Thus, good architectural practice helps a company innovate and change by providing both stability and flexibility. The insights provided by an enterprise architecture are needed on the one hand in determining the needs and priorities for change from a business perspective, and on the other hand in assessing how the company may benefit from technological and business innovations.

EXTERNAL DRIVERS

Next to the internal drive to execute effectively an organisation's strategy and optimise its operations, there are also external pressures that push organisations towards adopting enterprise architecture practice. They are customers, suppliers, other business partners, and regulatory bodies. For instance, the regulatory framework increasingly demands that companies and governmental institutions prove that they have a clear insight into their operations and that they comply with the applicable laws on, say, financial transactions.

SUMMARY

Architecture is the art and science of designing complex structures. Enterprise architecture, more specifically, is defined as a coherent whole of principles, methods, and models that are used in the design and realisation of an enterprise's organisational structure, business processes, information systems, and infrastructure. Architecture models, views, presentations, and analyses all help to bridge the 'communication gap' between architects and stakeholders. Architecture is an indispensable instrument in controlling the complexity of the enterprise and its processes and systems. On the one hand, we see internal drivers for using an architectural approach, related to the strategy execution of an organisation. Better alignment between business and IT leads to

lower cost, higher quality, better time-to-market, and greater customer satisfaction. On the other hand, external drivers from regulatory authorities and other pressures necessitate companies to have a thorough insight into their structure and operations. All of these drivers make a clear case for the use of enterprise architecture.

SERVICE ORIENTED ARCHITECTURE

Service-oriented architecture (SOA) is a software development model that allows services to communicate across different platforms and languages to form applications. In SOA, a service is a self-contained unit of software designed to complete a specific task. Service-oriented architecture allows various services to communicate using a loose coupling system to either pass data or coordinate an activity.

Loose coupling refers to a client of a service remaining independent of the service it requires. In addition, the client -- which can also be a service -- can communicate with other services, even when they are not related.

SOA benefits organizations by creating interoperability between apps and services. SOA will also ensure existing applications can be easily scaled, while simultaneously reducing costs related to the development of business service solutions.

The defining concepts of SOA are:

- The business value is more important than the technical strategy.
- The strategic goals are more important than benefits related to specific projects.
- Basic interoperability is more important than custom integration.
- Shared services are more important than implementations with a specific purpose.
- Continued improvement is more important than immediate perfection.

Service-oriented architecture is an implementation of the "service concept" or "service model" of computing. In this architectural style, business processes are implemented as software services, accessed through a set of strictly defined application program interfaces (APIs) and bound into applications through dynamic service orchestration.

Service-Oriented Architecture (SOA) Principles

There are 9 types of SOA design principles which are mentioned below:

1. **Standardized Service Contract**
   Services adhere to a service description. A service must have some sort of description which describes what the service is about. This makes it easier for client applications to understand what the service does.
2. **Loose Coupling**
   Less dependency on each other. This is one of the main characteristics of web services which just states that there should be as less dependency as possible between the web services and the client invoking the web service. So, if the service functionality changes at any point in time, it should not break the client application or stop it from working.

3. **Service Abstraction**
   Services hide the logic they encapsulate from the outside world. The service should not expose how it executes its functionality; it should just tell the client application on what it does and not on how it does it.

4. **Service Reusability**
   Logic is divided into services with the intent of maximizing reuse. In any development company re-usability is a big topic because obviously one wouldn't want to spend time and effort building the same code again and again across multiple applications which require them. Hence, once the code for a web service is written it should have the ability work with various application types.

5. **Service Autonomy**
   Services should have control over the logic they encapsulate. The service knows everything on what functionality it offers and hence should also have complete control over the code it contains.

6. **Service Statelessness**
   Ideally, services should be stateless. This means that services should not withhold information from one state to the other. This would need to be done from either the client application. An example can be an order placed on a shopping site. Now you can have a web service which gives you the price of a particular item. But if the items are added to a shopping cart and the web page navigates to the page where you do the payment, the responsibility of the price of the item to be transferred to the payment page should not be done by the web service. Instead, it needs to be done by the web application.

7. **Service Discoverability**
   Services can be discovered (usually in a service registry). We have already seen this in the concept of the UDDI, which performs a registry which can hold information about the web service.

8. **Service Composability**
   Services break big problems into little problems. One should never embed all functionality of an application into one single service but instead, break the service down into modules each with a separate business functionality.

9. **Service Interoperability**
   Services should use standards that allow diverse subscribers to use the service. In web services, standards as XML and communication over HTTP is used to ensure it conforms to this principle.

Major objectives of SOA

There are three major objectives of SOA; each objective focuses on a different part of the application lifecycle.

The first objective aims to structure procedures or software components as services. These services are designed to be loosely coupled to applications, so they are only used when needed. They are also designed to be easily utilized by software developers, who have to create applications in a consistent way.

The second objective is to provide a mechanism for publishing available services, which includes their functionality and input/output (I/O) requirements. Services are published in a way that allows developers to easily incorporate them into applications.

The third objective of SOA is to control the use of these services to avoid security and governance problems. Security in SOA revolves heavily around the security of the individual

components within the architecture; identity and authentication procedures related to those components; and securing the actual connections between the components of the architecture.

Implementation
SOA is independent of vendors and technologies. This means a wide variety of products can be used to implement the architecture. The decision of what to use depends on the end goal of the system.

SOA is typically implemented with web services such as simple object access protocol (SOAP) and web services description language (WSDL). Other available implementation options include:

- Windows Communication Foundation (WCF);
- gRPC; and
- messaging -- such as with Java Message Service (JMS), ActiveMQ and RabbitMQ.

SOA implementations can use one or more protocols and may also use a file system tool to communicate data. The protocols are independent of the underlying platform and programming language. The key to a successful implementation is the use of independent services that perform tasks in a standardized way, without needing information about the calling application, and without the calling application requiring knowledge of the tasks the service performs.

Common applications of SOA include:

- SOA is used by numerous armies and air force to deploy situational awareness systems.
- Healthcare organizations use SOA to improve healthcare delivery.
- SOA allows Mobile apps and games to use the mobile device's built-in functions, such as GPS.
- Museums use SOA to create virtualized storage systems for their information and content.

Benefits of Service-Oriented Architecture
SOA retains the procedure-call model commonly used in structured programming and standardizes the way in which business processes are automated and used, doing so in a way that maintains security and governance.

Other benefits of SOA include:

- **Reusable.** Services can be reused to make multiple applications. This is facilitated by the fact that SOA services are held in a service repository and linked on demand, making each service a generalized resource available to all -- subject to governance constraints. Reusing services allows organizations to save time and lower costs associated with development.
- **Easily maintained.** Since all services are independent, they can be easily modified and updated without affecting other services. This will also lower an organization's operating costs.
- **Promotes interoperability.** The use of a standardized communication protocol allows platforms to easily transmit data between clients and services regardless of the languages they're built on.

- **High availability.** SOA facilities are available to anyone upon request.
- **Increased reliability.** SOA generates more reliable applications because it is easier to debug small services than large code.
- **Scalable.** SOA allows services to run on different servers, thus increasing scalability. In addition, using a standard communication protocol allows organizations to reduce the level of interaction between clients and services. Lowering the level of interaction allows applications to be scaled without adding extra pressure.

SOA disadvantages

The WS model of SOA has never been widely accepted or adopted, in part because of its inherent complexity, but also because of the incompatibility between the WS approach and the RESTful API model of the internet. The internet, and the related cloud computing model, exposed specific issues with SOA/WS, and overall, the industry has moved to a different model.

Other disadvantages of SOA include:

- Implementation of SOA requires a large initial investment.
- Service management is complicated since the services exchange millions of messages that are hard to track.
- The input parameters of services are validated every time services interact, thus decreasing performance and increasing load and response times.

ENTERPRISE ARCHITECTURE FRAMEWORKS

Before delving into the details of ERP systems, we will quickly review the evolution of enterprise systems in organizations. Business organizations have become very complex. This is due to an increased layer of management hierarchy and an increased level of coordination across departments. Each staff role and management layer have different information needs and requirements. As such, no single information system can support all the business needs. Figure 1-1 shows the typical levels of management and corresponding information needs.

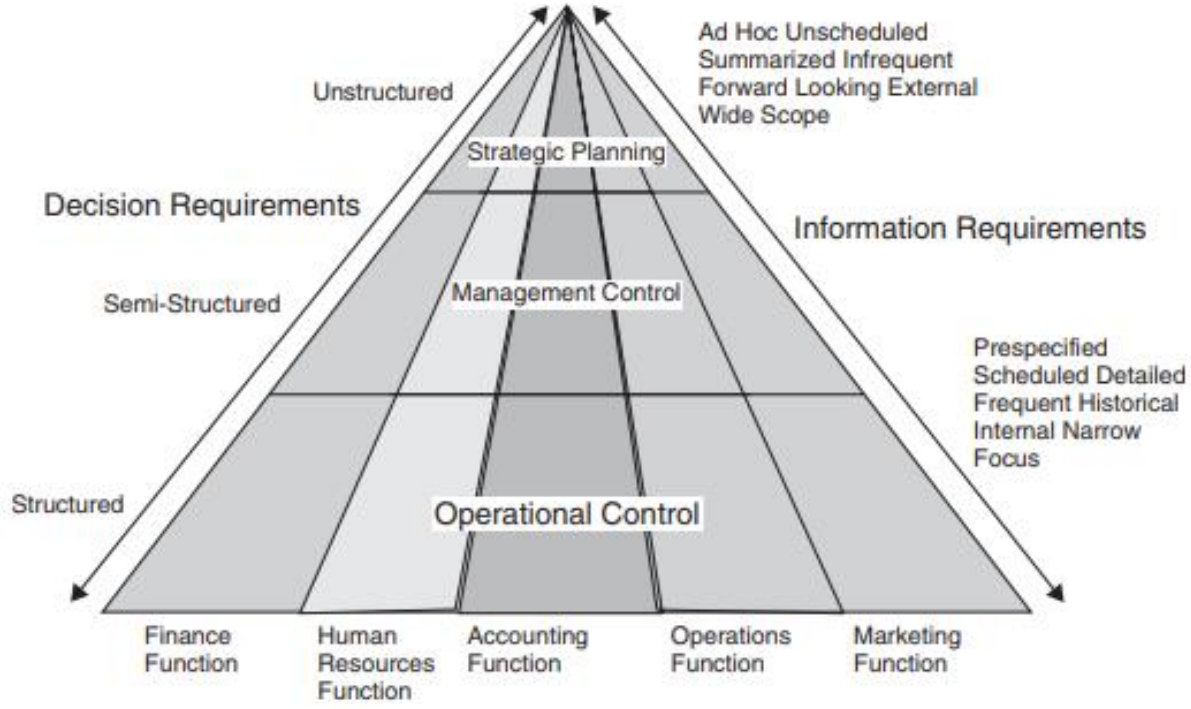

FIGURE 1-1 Management Pyramid with Information Requirements

Management is generally categorized into three levels: strategic, middle or mid-management, and operational.

At the strategic level, functions are highly unstructured and resources are undefined, whereas functions are highly structured and resources are predefined at the operational level. The mid-management level is somewhere in between depending on the hierarchy and organizational size. The pyramid shape in Figure 1-1 illustrates the information needs at each level of management.

The quantitative requirements are much less at the strategic level than they are at the operational level; however, the quality of information needed at the top requires sophisticated processing and presentation. The pyramid should assess and display the performance of the entire organization. For example, the CEO of a company may need a report that quickly states how a particular product is performing in the market vis-à-vis other company products over a period of time and in different geographical regions. Such a report is not useful to an operations manager, who is more interested in the detailed sales report of all products he or she is responsible for in the last month. The pyramid therefore suggests that managers at the higher level require a smaller quantity of information, but that it is a very high quality of information.

On the other hand, the operational-level manager requires more detailed information and does not require a high level of analysis or aggregation as do their strategic counterparts. Today's enterprise systems are designed to serve these varied organizational requirements.

Enterprise systems, therefore, are a crucial component of any successful organization today. They are an integral part of the organization and provide computer automation support for most business functions such as accounting, finance, marketing, customer service, human resource management, operations, and more. In general, they play a critical role in both the primary and secondary activities of the organization's value chain.

INFORMATION SILOS AND SYSTEM INTEGRATION

As organizations become larger and more complex, they tend to break functions into smaller units by assigning a group of staff to specialize in these activities. This allows the organization to manage complexity as well as some of the staff to specialize in those activities to enhance productivity and efficiency. The role of information systems has been and always will be one of supporting business activities and enhancing the workers, efficiency. Over time, however, as business changes and expands, systems need to change to keep pace. The result is sometimes a wide variety of information systems and computer architecture configurations, which creates a hodgepodge of independent non-integrated systems. These systems ultimately create bottlenecks and interfere with productivity.

In today's globally competitive environment, an organization will find it very difficult to operate and survive with silo information systems. Organizations need to be agile and flexible, and will require the same from their information systems. These systems need to have integrated data, applications, and resources from across the organization. Integrated information systems are needed today to focus on customers, to process efficiency, and to help build teams that bring employees together that cross functional areas.

To compete effectively in today's market, organizations have to be customer focused and cost efficient. This demands cross-functional integration among the accounting, marketing, and other departments of the organization. This has led to the creation of business units (BU) within organizations that integrate personnel from the various functional units to work together on a variety of projects within an organization. Business units are dynamic suborganizations created and eliminated depending on need. BUs can be in existence for a few weeks or a few years, which makes it impossible physically to locate the personnel in an adjacent geographical space. This demands that the information systems be flexible and fluid across the departmental boundaries. In addition, it requires that systems are accessible anyplace and anytime. These business requirements ultimately created the need for enterprise systems to support the multifunctional needs of the organization.

ENTERPRISE RESOURCE PLANNING SYSTEMS

What Is An ERP? Enterprise resource planning (ERP) systems are the specific kind of enterprise systems to integrate data across and be comprehensive in supporting all the major functions of the organization.
The enterprise resource planning (ERP) system (sometimes just called enterprise software) was developed to bring together an entire organization in one software application. Simply put, an ERP system is a software application utilizing a central database that is implemented throughout the entire organization. Let's take a closer look at this definition:

- "A software application": An ERP is a software application that is used by many of an organization's employees.

- "utilizing a central database": All users of the ERP edit and save their information from the data source. What this means practically is that there is only one customer database, there is only one calculation for revenue, etc.

- "that is implemented throughout the entire organization": ERP systems include functionality that covers all of the essential components of a business. Further, an organization can purchase modules for its ERP system that match specific needs, such as manufacturing or planning,

ERPs, shown in Figure 1-2, are basically integrated information systems that support such enterprise functions as accounting, financial, marketing, and production requirements of

organizations. This allows for real-time data flows between the functional applications. ERP systems are comprehensive software applications that support critical organizational functions.

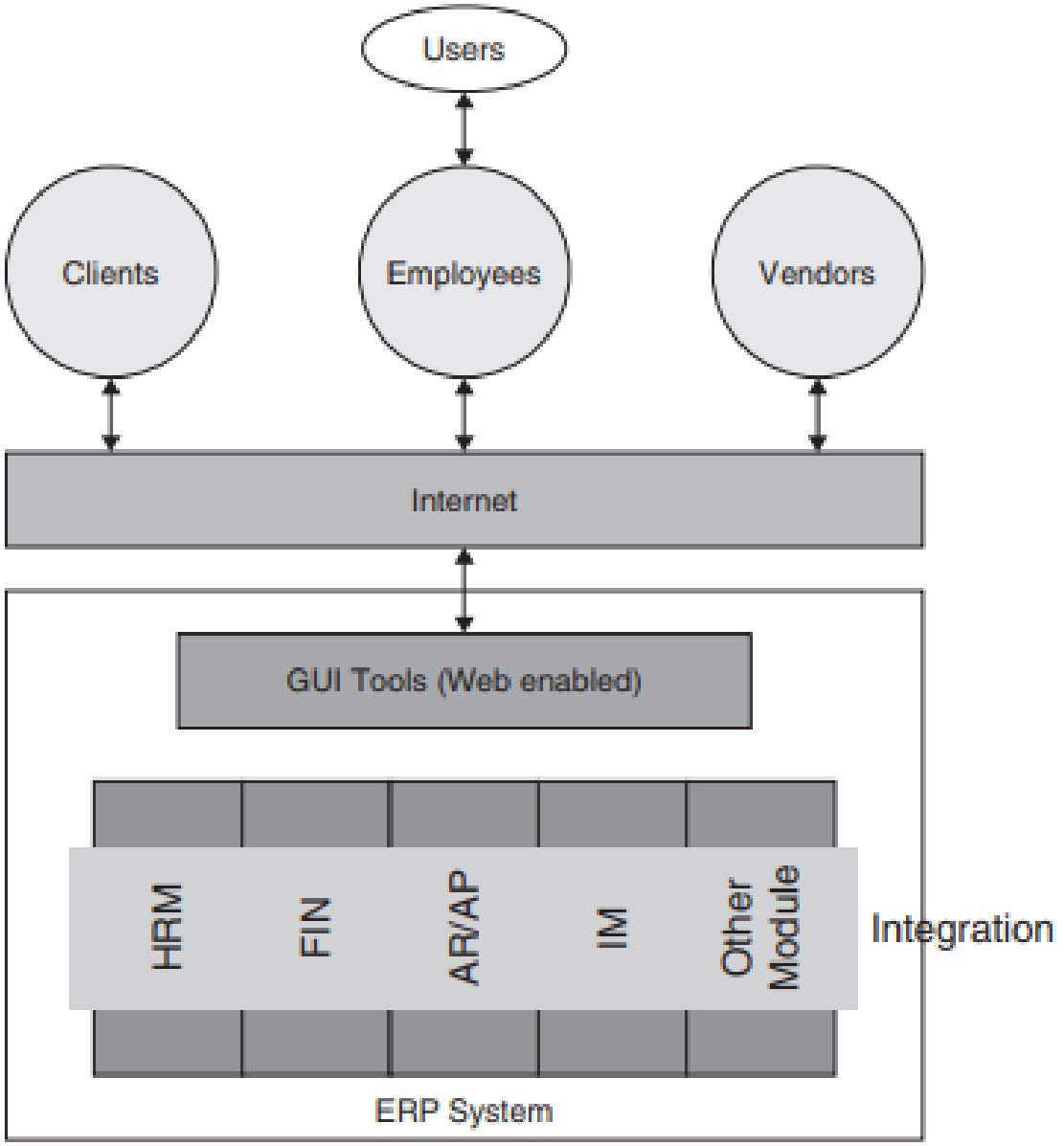

FIGURE 1-2 Integrated Systems—ERP

As shown in Figure 1-2, they integrate both the various functional aspects of the organization and the systems within the organization with those of its partners and suppliers. Furthermore, these systems are "Web enabled," meaning that they work using Web clients, making them accessible to all of the organization's employees, clients, partners, and vendors from anytime and anyplace, thereby promoting the BUs' effectiveness.

ERP system's goal is to make information flow be both dynamic and immediate, therefore increasing the usefulness and value of the information. In addition, an ERP system acts as a central repository eliminating data redundancy and adding flexibility. A few of the reasons companies choose to implement ERP systems is the need to "increase supply chain efficiency, increase customer access to products and services, reduce operating costs, respond more rapidly to a changing marketplace, and extract business intelligence from the data.
Another goal of ERP system is to integrate departments and functions across an organization onto a single infrastructure that serves the needs of each department. This is a difficult, if not an impossible, task considering that employees in the procurement department will have very different needs than will employees in the accounting department. Each department historically

has its own computer system optimized for the particular ways that the department does its work.

An ERP system, however, combines them all together into a single, integrated software environment that works on a single database, thereby allowing various departments to share information and communicate with each other more easily. To achieve this high level of integration, however, departments may sometimes give up some functionality for the overall benefit of being integrated. The central idea behind data integration is that clean data can be entered once into the system and then reused across all applications.

In summary, ERP systems are the mission-critical information systems in today's business organization. They replace an assortment of systems that typically existed in those organizations (e.g., accounting, finance, HR, transaction processing systems, materials planning systems, and management information systems). In addition, they solve the critical problem of integrating information from various sources inside and outside the organization's environment and make it available, in real time, to all employees and partners of the organization.

EVOLUTION OF ERP

During the 1960s and 1970s, most organizations designed silo systems for their departments. As the production department grew bigger, with more complex inventory management and production scheduling, they designed, developed, and implemented centralized production systems to automate their inventory management and production schedules. These systems were designed on mainframe legacy platforms using such programming languages as COBOL, ALGOL, and FORTRAN. The efficiencies generated with these systems saw their expansion to the manufacturing area to assist plant managers in production planning and control. This gave birth to material requirements planning (MRP) systems in the mid-1970s, which mainly involved planning the product or parts requirements according to the master production schedule.

Later, the manufacturing resources planning (MRP II) version was introduced in the 1980s with an emphasis on optimizing manufacturing processes by synchronizing the materials with production requirements. MRP II included such areas as shop floor and distribution management, project management, finance, job-shop scheduling, time management, and engineering. ERP systems first appeared in the early 1990s to provide an integrated solution to the increased complexity of businesses and support enterprise to sustain their compatibility in the emerging dynamic global business environment. Built on the technological foundations of MRP and MRP II, ERP systems integrated business processes across both the primary and secondary activities of the organization's value chain, including manufacturing, distribution, accounting, finances, human resource management, project management, inventory management, service and maintenance, and transportation. ERP systems' major achievement was to provide accessibility, visibility, and consistency across all functions of the enterprise.3 ERP II systems today have expanded to integration of interorganizational systems providing back-end support for such electronic business functions as business-to-business (B2B) and electronic data interchange (EDI). From the technological platform perspective, therefore, ERPs have evolved from mainframe and centralized legacy applications to more flexible, tiered client–server architecture using the Web platform. Table 1-1 summarizes the evolution of ERP from 1960s to 2000s.

TABLE 1-1 Evolution of Enterprise Systems

Timeline	System	Platform	Description
1960s	Inventory management and control	Mainframe legacy using third-generation software (e.g., Cobol, Fortran)	With a focus on efficiency, these systems were designed to manage and track inventory of raw materials and guide plant supervisors on purchase orders, alerts, and targets, providing replenishment techniques and options, inventory reconciliation, and inventory reports.
1970s	Material requirements planning (MRP)	Mainframe legacy using third-generation software (e.g., Cobol, Fortran)	With a focus on sales and marketing, these systems were designed for job shop scheduling processes. MRP generates schedules for production planning, operations control, and inventory management.
1980s	Manufacturing requirements planning (MRP II)	Mainframe legacy using fourth-generation database software and manufacturing applications	With a focus on manufacturing strategy and quality control, these systems were designed for helping production managers in designing production supply chain processes—from product planning, parts purchasing, inventory control, and overhead cost management to product distribution.
1990s	Enterprise resource planning (ERP)	Mainframe or client–server using fourth-generation database software and package software application to support most organizational functions	With a focus on application integration and customer service, these systems were designed for improving the performance of the internal business processes across the complete value chain of the organization. They integrate both primary business activities like product planning, purchasing, logistics control, distribution, fulfillment, and sales; additionally, they integrate secondary or support activities like marketing, finance, accounting, and human resources.
2000s	Extended ERP or ERP II	Client–server using Web platform, open source and integrated with fifth-generation applications like SCM, CRM, SFA. Also available on Software as a Service (SaaS) environments	With a focus on agility and customer-centric global environment, these systems extended the first-generation ERP into interorganizational systems ready for e-Business operations. They provide anywhere anytime access to resources of the organization and their partners; additionally, they integrate with newer external business modules such as supply chain management, customer relationship management, sales force automation (SFA), advanced planning and scheduling (APS), etc.

ERP SYSTEM COMPONENTS

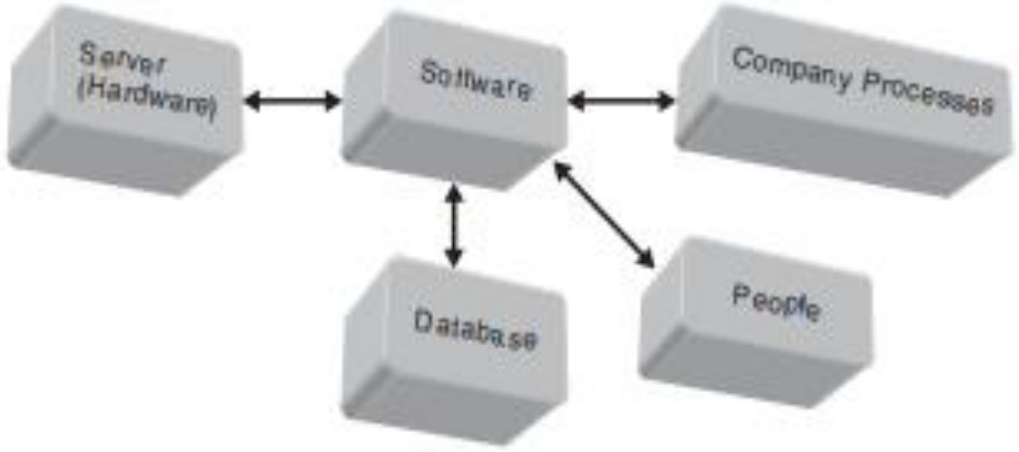

FIGURE 1-3 ERP Components

As shown in Figure 1-3, an ERP system, like its information system counterpart, has similar components such as hardware, software, database, information, process, and people.

These components work together to achieve an organization's goal of enhanced efficiency and effectiveness in their business processes. An ERP system depends on hardware (i.e., servers and peripherals), software (i.e., operating systems and database), information (i.e., organizational data from internal and external resources), process (i.e., business processes, procedures, and policies), and people (i.e., end users and IT staff) to perform the input, process, and output phases of a system.

The basic goal of ERP, like any other information system, is to serve the organization by converting data into useful information for all the organizational stakeholders. The key components for an ERP implementation are hardware, software, database, processes, and people. These components must work together seamlessly for the implementation to be successful. The implementation team must carefully evaluate each component in relation to the others while developing an implementation plan. Hardware, software, and data play a significant role in an ERP system implementation. Failures are often caused by a lack of attention to the business processes and people components. Both people involvement and process integration will need to be addressed from the very early stages in the implementation plan. Staff must be allowed to play a key role in the project from the beginning.

As shown in Figure 1-4, each component must be layered appropriately and each layer must support the efficiency of the other layers. The layered approach also provides the ability to change layers without significantly affecting the other layers. This can help organizations lower the long-term maintenance of the ERP application because changes in one layer do not necessarily require changes in other layers.

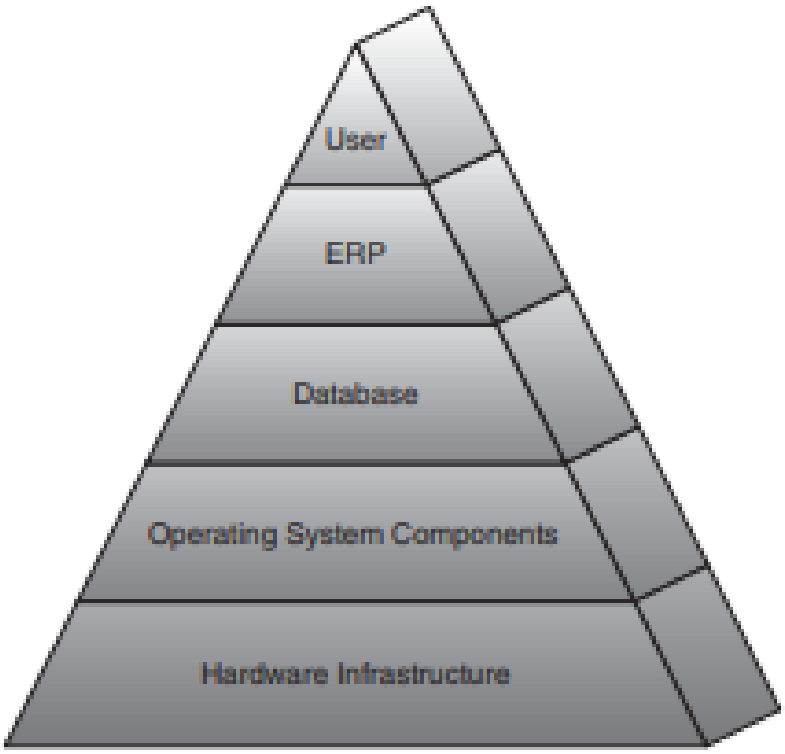

FIGURE 1-4 ERP Components Integration

ERP ARCHITECTURE

The architecture of the ERP implementation influences the cost, maintenance, and the use of the system. A flexible architecture is best because it allows for scalability as the needs of the organization change and grow. A system's architecture is a blueprint of the actual ERP system and transforms the high-level ERP implementation strategy into an information flow with interrelationships in the organization. The ERP architecture helps the implementation team build the ERP system for an organization. The role of system architecture is similar to the architecture of a home, which takes the vision of the homeowners with the system components similar to the wiring, plumbing, and furnishings of a home. The process of designing ERP system architecture is slightly different from other IT architectures. Whereas other IT architectures are driven by organizational strategy and business processes, if purchased, ERP architecture is often driven by the ERP vendor. This is often referred to as package-driven architecture. The reason for this reversal is that most ERP vendors claim to have the best practices of their industry's business processes captured in their system logic. This argument has proven very powerful in convincing organizations to spend millions of dollars for the ERP package. In order to leverage this investment and maximize the return on investment, an ERP implementation is driven by the requirements contained in the package.

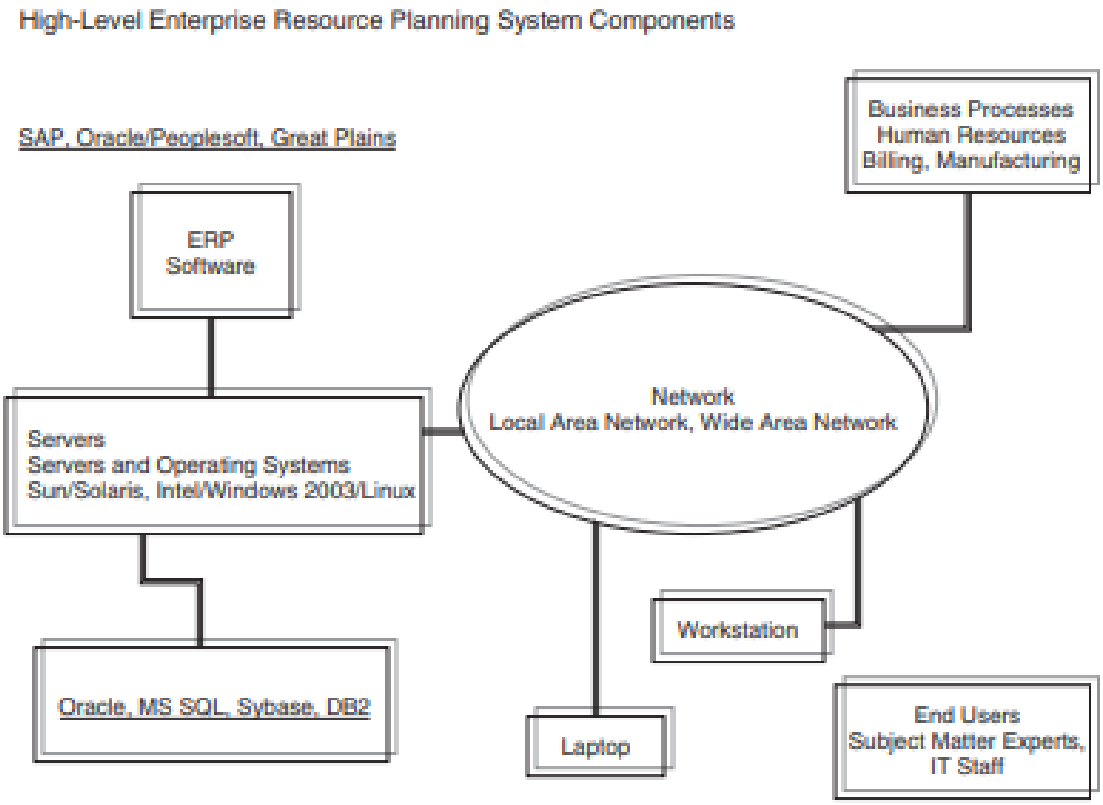

FIGURE 1-5 Example of Architecture of ERP at Large University

The architecture must therefore be conceived after the selection of ERP software, whereas the architecture is conceived well before buying or developing software in other IT implementations. An ERP package can have a very different implementation outcome from one organization to another. In the architecture of a large university, an ERP system can be very complex and must be designed and tested thoroughly before implementing it in the organization (Figure 1-5). The architecture sets the stage for modifications or customizations to support an organization's policies and procedures, data conversion, system maintenance, upgrades, backups, security, access, and controls. Many organizations often make the mistake of ignoring the system architecture stage and jumping directly into ERP implementation because they have planned a "vanilla" or "as-is" implementation. This can be disastrous because the organization will not be prepared for long-term maintenance and upkeep of the system.

The two types of architectures for an ERP system are logical (see Figure 1-6) and physical or tiered (see Figure 1-7). The logical architecture, shown in Figure 1-6, focuses on supporting the requirements of the end users, whereas the physical architecture focuses on the efficiency (cost, response time, etc.) of the system. The logical architecture provides the database schemas of entities and relationships at the lowest tier, followed by the core business processes and business logic handled by the system at the second tier. The third tier provides details on the applications that support the various business functions built in to the ERP system. The end users do not ever see the first and second tiers because they interact primarily with the client–user interface application tier that provides them access to the functional applications

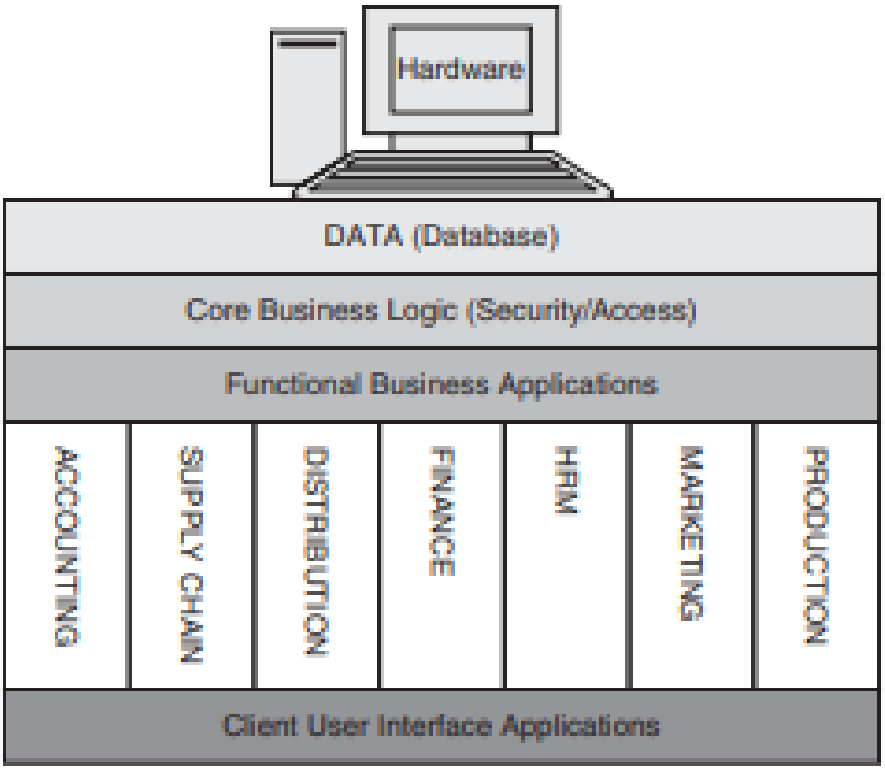

FIGURE 1-6 Logical Architecture of an ERP System

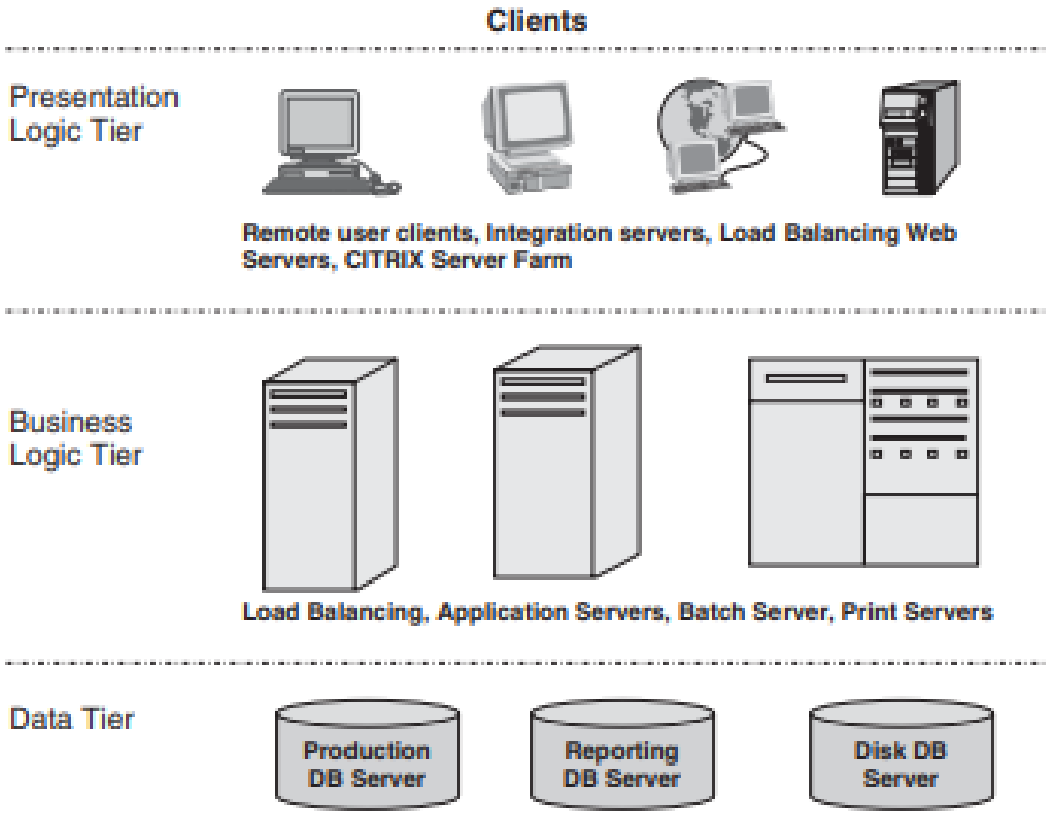

FIGURE 1-7 Tiered Architecture Example of ERP System

e-Business and ERP

Both e-Business and ERP technologies have pretty much evolved simultaneously, since the 1990s. Hence, during the early days, many people thought only one would survive for the long term. With the initial enthusiasm and support for e-Business, many analysts predicted the doom of ERP. Instead, both have flourished beyond everyone's expectation from those early days. One reason for their success is this simultaneous growth. The early predictions were based on the assumptions that these two technologies were competing for the same market. Yes there are some similarities between the two, namely:

- both provide platform for systems integration or data sharing. While e-Business systems are better for sharing unstructured data and collaboration, ERP are better for sharing structured or transaction data; also, e-Business focus was on external integration (interorganizational), while ERP systems' initial focus was on internal data integration. Therefore, e-Business and ERP are more of complementary technologies (see Figure 1-8) rather than competing technologies as predicted earlier.

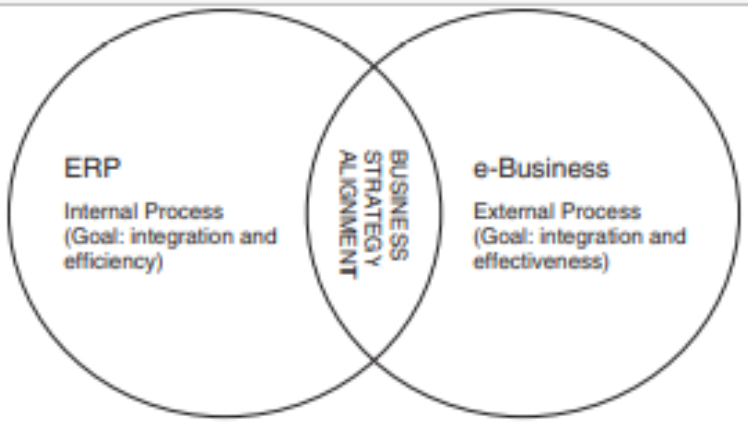

FIGURE 1-8 e-Business and ERP

The major reasons for these complementary technologies are listed below:

1. e-Business technology focus has been on linking a company with its external partners and stakeholders, whereas ERP focus has been on integrating the functional silos of an organization into an enterprise application. e-Business technologies that have emerged as successful over the decade (e.g., business-to-consumer and business-to-business) have generally focused on market growth by selling products and services to new consumers and markets. On the other hand, ERP technology has been successful in integrating business processes across the functional spectrum of the organization and in providing a central repository of all corporate data, information, and knowledge, thereby increasing organizational efficiency and worker productivity.

2. e-Business is a disruptive technology, whereas ERP is adaptive technology. e-Business practically transformed the way business operates in terms of buying and selling, customer service, and its relationships with suppliers. This caused a lot of disruptions in organizational strategy, structure, power, and the like. ERP has emerged as an adapter by merging the early data processing and integration efforts within a large corporation. It has been very successful in aligning and integrating accounting, finance, human resource, and manufacturing technologies by aligning business processes with information processing logic and in transforming these organizations from pure hierarchical structures to matrix and other hybrid or flexible organizational structures. Thus, even though e-Business caused a lot of disruptions in business, ERP helped these businesses survive by allowing them to adapt quickly to these disruptions.
3. Finally, the early focus of e-Business was on communication (e-mail), collaboration (calendaring, scheduling, group support), marketing and promotion (Web sites), and

electronic commerce. These can all be considered front-office functions that involve user and/or customer interactions. In contrast, the focus of ERP systems was mainly on data sharing, systems integration, business process change, and improving decision making through the access of data from a single source. These functions can all be considered back-office functions helping the operational efficiencies of employees, vendors, and suppliers. For example, e-commerce, which facilitates selling of products online, requires tremendous back-end support, namely, fulfillment of the online order. This task can be efficient if an organization has an ERP system in place.

BENEFITS AND LIMITATIONS OF ERP

ERP systems require a substantial investment from an organization in terms of cost, time, and people. These investments can run into millions of dollars over several years and involve hundreds of people from the organization. No organization will be willing to invest a huge amount of resources unless the benefits outweigh the costs.

The benefits and limitations of ERP can be looked at from a systems and business viewpoint; similarly, like other IT projects, the returns can be tangible and intangible, as well as short term and long term. The management within an organization that implements an ERP system has to account for the benefits and limitations of this system from all viewpoints and focus on the big picture to justify the huge investments in this system to the stakeholders. A strong commitment from management is critical for the success of ERP systems. This commitment will not be internalized unless a thorough analysis of benefits and limitations is communicated.

The system benefits of ERP systems are as follows:

• Integration of data and applications across functional areas of the organization (i.e., data can be entered once and used by all applications in the organization, improving accuracy and quality of the data).

• Maintenance and support of the system improves as the IT staff is centralized and is trained to support the needs of users across the organization.

• Consistency of the user interface across various applications means less employee training, better productivity, and cross-functional job movements.

• Security of data and applications is enhanced due to better controls and centralization of hardware, software, and network facilities.

• Increasing agility of the organization in terms of responding to changes in the environment for growth and maintaining the market share in the industry.

• Sharing of information across the functional departments means employees can collaborate easily with each other and work in teams.

• Linking and exchanging information in real time with its supply chain partners can improve efficiency and lower costs of products and services.

• Quality of customer service is better and quicker as information flows both up and down the organization hierarchy and across all business units.

• Efficiency of business processes are enhanced due to business process reengineering of organization functions. • Retraining of all employees with the new system can be costly and time consuming.

The system limitations of ERP systems are as follows:

• Complexity of installing, configuring, and maintaining the system increases, thereby requiring specialized IT staff, hardware, network, and software resources.

• Consolidation of IT hardware, software, and people resources can be cumbersome and difficult to attain.

• Data conversion and transformation from an old system to a new system can be an extremely tedious and complex process.

• Retraining of IT staff and personnel to the new ERP system can produce resistance and reduce productivity over a period of time

• Change of business roles and department boundaries can create upheaval and resistance to the new system.

• Reduction in cycle time in the supply chain from procurement of raw materials to production, distribution, warehousing, and collection.